



IntelliQR

Complete Project Documentation: working, architecture, file structure, technology stack, setup, and start commands

React 19

TypeScript

FastAPI

MongoDB

Local-first

QR Platform

Repository: IntelijQR

Document version: 1.0 | **Prepared:** 10 August 2026

Scope: frontend, backend API, persistence, local file storage, security, and tests

Contents

1	Project overview and goals
2	How the project works end to end
3	Feature-by-feature working
4	System architecture and data flow
5	Frontend structure and responsibilities
6	Backend structure and responsibilities
7	Technology stack
8	Security, persistence, and privacy
9	Setup and start commands
10	API and route reference
11	Testing, build, and troubleshooting

1. Project overview

IntelliQR is a local-first, full-stack QR management platform. It creates styled QR codes, decodes uploaded or webcam QR images, manages dynamic destinations, records privacy-conscious analytics, generates QR codes in bulk, and provides encrypted access-controlled sharing for links and files.

The application is designed as a portfolio-grade multi-user system rather than a single-page QR utility. It includes authentication, owner-isolated records, role-based administration, notifications, profile settings, English/Hindi/Gujarati interface support, and automated backend and frontend tests.

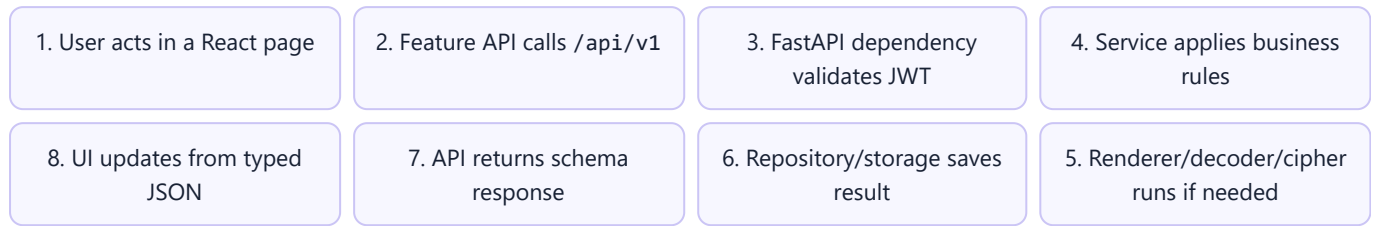
Core idea: the React application presents the user interface, the FastAPI service enforces business rules and security, MongoDB stores records and events, and local directories store generated images, ZIP archives, and encrypted shared files.

Main user groups

- **Public visitors:** view the landing page, register/login, follow dynamic QR redirects, and complete protected link/file access checks.
- **Members:** generate, scan, manage, export, analyze, bulk-create, and securely share QR-based content.
- **Administrators:** view platform statistics, search users, change roles/status, and inspect audit activity.

2. How the project works end to end

Normal authenticated request



The frontend uses one API client. It sends cookies on every request, adds the in-memory access token, converts error envelopes to `ApiError`, and performs a single refresh-and-retry when a protected request returns HTTP 401.

Authentication lifecycle

1. Registration or login verifies credentials and returns a short-lived JWT access token.
2. The access token is kept only in JavaScript memory. The refresh token is held in an `HttpOnly` backend cookie.
3. On page restoration or an expired access token, `POST /auth/refresh` rotates the refresh session and supplies a new access token.
4. Logout, password rotation, account disablement, or security-sensitive role changes invalidate sessions. A token-version check protects every authenticated request.

Dynamic QR lifecycle

1. A URL QR generation request builds a stable backend URL such as `/r/{slug}`, rather than permanently embedding the final destination.
2. The renderer creates PNG, SVG, and PDF files from the persisted BrandCraft design.
3. When scanned, the redirect endpoint checks status, expiry, scan limit, and access policy.
4. A permitted scan atomically increments usage, records a privacy-safe analytics event, and redirects to the current destination.
5. Because the QR contains a stable slug, the owner can change or pause the destination without printing a new code.

3. Feature-by-feature working

Feature	Working
QR Generator + BrandCraft	Creates URL, text, email, phone, Wi-Fi, contact, and location payloads. Users control colors/gradients, modules, frame, correction level, size, margin, and validated logos. Generated items support CRUD, search, filters, favourites, pause/activate, limits, expiry, and PNG/SVG/PDF download.
SafeScan	Accepts PNG/JPEG/WebP uploads or webcam frames up to 10 MB. OpenCV/pyzbar decode one or more values. SmartClassifier identifies structured content and SafeScan applies local URL risk heuristics. The source image is not retained.
Analytics	Uses successful dynamic redirect events to calculate totals, unique-visitor estimates, trends, device/browser/OS/location breakdowns, top QR codes, and recent scans. Period and QR filters are supported.
SecureVault	Protects dynamic destinations with public, password, signed-in-member, or private email-allowlist policies. Destinations are encrypted at rest. Successful access produces a short-lived, slug-bound JWT grant before redirect.
BulkForge	Parses CSV/XLSX files (up to 250 rows), validates each row through the normal generator, tracks queued/processing/completed/partial/failed state, preserves bounded row errors, and packages successful PNG/SVG/PDF outputs in an owner-only ZIP.
ShareVault	Signature-checks approved files up to 25 MB, encrypts them with AES-256-GCM, generates a recipient QR, and enforces policy, expiry, pause, and atomic download limits. Files are decrypted in memory only after a short-lived access grant.
Dashboard	Aggregates owner-scoped QR, favourite, scan, share, bulk, and export statistics plus recent activity and download history.
Notifications	Stores security, scan-limit, ShareVault, and BulkForge events; exposes unread counts and read/delete/preferences controls. Optional email is deliberately restricted to a localhost SMTP capture server.
Profile + i18n	Updates display name, password, theme, and locale. Shared dictionaries provide English, Hindi, and Gujarati UI content, and locale survives session restoration.
Administration	Role-gated views expose platform counts, paginated user management, role/status controls, and privacy-safe audit history. Rules prevent self-lockout and removal of the last active administrator.

4. System architecture

Layer	Components	Responsibility
Presentation	React pages, layouts, UI components	Navigation, forms, visualization, public/protected pages, accessibility feedback.
Frontend domain	features/* APIs, hooks, types, contexts	Typed feature boundaries, stateful workflows, auth/session and locale state.
HTTP API	FastAPI routers, endpoints, Pydantic schemas	Versioned contract, validation, dependency injection, status codes, responses.
Application services	Auth, generator, scanner, vault, analytics, bulk, share, admin	Business rules and orchestration independent of HTTP presentation.
Persistence	MongoDB repositories	Owner-scoped documents, sessions, jobs, notifications, events, atomic counters.
Infrastructure	QR renderer/decoder, parsers, ZIP/file storage, cipher, local SMTP	External-format processing and local resource access.

Runtime topology

```
Browser at http://127.0.0.1:5173
|  JSON / multipart / cookie + Bearer JWT
v
FastAPI at http://127.0.0.1:8000
|-- /api/v1/*   versioned application API
|-- /r/{slug}  dynamic QR redirect
|-- /docs      Swagger UI
|
+--> MongoDB at mongodb://localhost:27017 (database: intelliqr)
+--> ../data/generated  rendered QR files
+--> ../data/bulk       ZIP job archives
+--> ../data/shares     encrypted shared-file blobs
+--> localhost:1025     optional SMTP capture server
```

Backend request safeguards

Middleware assigns or preserves X-Request-ID, measures request time, logs completion, and adds API-version and content-type headers. Consistent JSON error envelopes are returned for application, validation, and unexpected errors. CORS allows the local Vite origins and credentials.

5. Frontend file structure

frontend/		VaultAccessPage.tsx	public link
package.json	scripts and	authorization	
dependencies		SharedFileAccessPage.tsx	public file
vite.config.ts	React +	access	
Tailwind plugins		ProfilePage.tsx	
tsconfig.json	TypeScript	account/settings	
configuration		NotificationsPage.tsx	local inbox
index.html	Vite HTML	AdminPage.tsx	admin control
entry		center	
src/		features/	
main.tsx	mounts	auth/	API, context,
providers and App		hook, types	
App.tsx	router and	qr-generator/	API, hooks,
lazy page routes		controls, preview	
index.css		qr-scanner/	API, hook,
Tailwind/global visual system		types	
components/		analytics/	API, hook,
ProtectedRoute.tsx	auth/role	chart components	
route guard		bulk-forge/	API, hook,
layout/	navbar,	types	
sidebar, page shells		secure-vault/	public access
ui/	button, input,	API/types	
card, badge, etc.		share-vault/	
pages/	route-level	owner/recipient APIs and hooks	
screens		personal-dashboard/	summary
LandingPage.tsx	public	API/hook/types	
introduction		notifications/	inbox
LoginPage.tsx	sign in	API/hook/types	
RegisterPage.tsx	create account	admin/	admin
DashboardPage.tsx	personal	API/hook/types	
summary		i18n/	locale
QrGeneratorPage.tsx	create/manage	context/dictionaries	
QR codes		lib/api/client.ts	shared fetch +
QrScannerPage.tsx	SafeScan	refresh logic	
workflow		hooks/useTheme.ts	local theme
AnalyticsPage.tsx	charts and	preference	
breakdowns		tests/	Vitest feature
BulkForgePage.tsx	batch jobs	API tests	
ShareVaultPage.tsx	owner file	dist/	production
sharing		build output	

Pages are lazy-loaded from App.tsx. Public routes are separate from the authenticated DashboardLayout; the admin route adds a second role guard.

6. Backend file structure

backend/		profile_service.py	account updates
requirements.txt	Python	dashboard_service.py	aggregation
dependencies		notification_service.py	
.env.example	safe	alerts/preferences	
configuration template		admin_service.py	admin
README.md / API.md	local and REST	invariants/auditing	
notes		qr/	
app/		generator_service.py	generation
main.py	app,	lifecycle	
middleware, errors, routers		scanner_service.py	decode
core/		analysis/history	
config.py	environment	payload_builder.py	content
settings		serialization	
security.py	passwords and	classifier.py	content
JWTs		classification	
vault.py	AES cipher and	safe_scan.py	local URL
grants		heuristics	
exceptions.py	application	vault_access_service.py	protected
error types		links	
logging.py	structured	bulk/bulk_service.py	batch
request logging		orchestration	
db/mongodb.py	connection and	share/share_service.py	encrypted
indexes		sharing	
api/		analytics/	aggregation +
dependencies.py		scan context	
repositories/services/auth	DI	infrastructure/	
v1/router.py	versioned	qr/	renderer and
router composition		decoder	
v1/endpoints/	HTTP endpoint	bulk/	CSV/XLSX
modules		parser, ZIP storage	
schemas/	Pydantic	storage/	QR and
request/response models		encrypted file storage	
models/	internal domain	notifications/	localhost SMTP
records		sender	
repositories/	MongoDB	scripts/	integration
persistence operations		smoke tests	
services/		tests/	Pytest
auth_service.py	auth/session	unit/API tests	
rules			

The dependency module constructs repositories and services for each request. This keeps endpoints thin: they validate HTTP input and delegate the actual rule enforcement to services.

7. Technology stack

Area	Technology	Purpose
UI framework	React 19.1, React DOM	Component-based browser application.
Language/build	TypeScript 5.9, Vite 7	Static types, development server, optimized production bundles.
Navigation	React Router DOM 7	Public, authenticated, lazy, and role-protected routes.
Styling/UI	Tailwind CSS 4, Radix UI, Lucide React, clsx, tailwind-merge	Responsive styling, accessible primitives, icons, class composition.
Browser QR	qrcode	Matching live QR preview while editing designs.
API framework	Python 3, FastAPI, Uvicorn	Async REST endpoints, dependency injection, OpenAPI server.
Validation/config	Pydantic 2, pydantic-settings	Typed schemas and environment-driven settings.
Database	MongoDB, PyMongo	Users, sessions, QR records, scan events, jobs, shares, notifications.
Security	PyJWT, pwdlib/bcrypt, cryptography	Access/refresh/grant JWTs, password hashing, AES-256-GCM encryption.
Image/QR	qrcode, Pillow, OpenCV, pyzbar	Render QR assets, validate images, and decode uploaded/webcam images.
Batch/files	openpyxl, Python CSV/ZIP/file APIs	CSV/XLSX imports and downloadable job archives.
Testing	Pytest, HTTPX/TestClient, Vitest	Backend behavior/contracts and frontend feature API tests.

Design choices

- **Local-first:** no cloud database, file store, translation provider, geolocation API, or external SMTP is required.
- **Feature-oriented frontend:** each domain owns its API wrapper, types, and hooks.
- **Layered backend:** endpoint, service, repository, and infrastructure responsibilities remain distinct.
- **Stable dynamic links:** analytics and destination updates work without regenerating printed QR codes.

8. Security, persistence, and privacy

Concern	Implementation
Passwords	Bcrypt-backed password service; plaintext passwords are never stored.
Browser tokens	Access token remains in memory; rotating refresh token uses an HttpOnly cookie.
Authorization	Every owner resource is scoped to the authenticated user; admin endpoints require a current database role.
Revocation	Refresh sessions plus token-version validation support logout, password rotation, role changes, and disabled accounts.
Protected data	SecureVault destinations and ShareVault files use AES-256-GCM with unique nonces. Shared passwords use hashes.
Temporary grants	Purpose-bound, short-lived JWTs bind access to one QR or file slug.
Scan privacy	Uploaded images are decoded without retention. Analytics store a keyed one-way visitor hash, not raw IP or full user-agent strings.
Redirect privacy	Protected redirects disable caching and referrer propagation to avoid leaking grant information.
Email safety	Only localhost SMTP hosts are accepted; notification failure never blocks the main operation.

Persisted data

MongoDB collections represented by the repository/model layer include users, refresh sessions, QR generations, scanner history, dynamic scan events, bulk jobs, shared files, share download events, notifications, and audit/dashboard-related events. Local generated artifacts live under the root `data/` directory, which is ignored by Git.

Production caution: replace both example secrets, use a dedicated vault encryption key, retain that key safely, set secure cookies behind HTTPS, restrict CORS, and configure `MONGODB_REQUIRED_ON_STARTUP=true`. Losing or changing the vault key makes existing protected data unreadable.

9. Setup and start commands

Prerequisites

- Python 3.11+ and Node.js/npm
- A local MongoDB server reachable at `mongodb://localhost:27017`
- Windows PowerShell (commands below); Git is recommended

One-time backend setup

```
cd C:\codes\react\IntelijQR\backend
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
Copy-Item .env.example .env
```

Edit `backend/.env` and replace `INTELIQOR_JWT_SECRET` and `INTELIQOR_VAULT_ENCRYPTION_KEY` with separate random values of at least 32 characters. Do not commit `.env`.

One-time frontend setup

```
cd C:\codes\react\IntelijQR\frontend
npm install
Copy-Item .env.example .env
```

Start the application (three terminals)

Terminal	Command
1 — MongoDB	Start the local MongoDB Windows service, or run your installed <code>mongod</code> instance.
2 — Backend	<pre>cd C:\codes\react\IntelijQR\backend .\.venv\Scripts\Activate.ps1 uvicorn app.main:app --reload</pre>
3 — Frontend	<pre>cd C:\codes\react\IntelijQR\frontend npm run dev</pre>

Open `http://127.0.0.1:5173`. Swagger API documentation is at `http://127.0.0.1:8000/docs`; readiness is at `http://127.0.0.1:8000/api/v1/health/ready`.

Optional administrator

```
INTELIQOR_ADMIN_EMAILS=["admin@example.com"]
```

Add the setting to `backend/.env`, restart the API, then register or log in with that email.

10. Route and API reference

Frontend routes

Access	Routes
Public	/, /login, /register, /access/:slug, /share/:slug
Authenticated	/dashboard, /generator, /scanner, /analytics, /bulk, /share-vault, /profile, /notifications
Admin	/admin
Compatibility	/qr-codes redirects to /generator

Backend resource groups under /api/v1

Prefix	Main operations
/auth	Register, login, rotate refresh token, logout.
/users	Current profile, display name, locale, password.
/qr/generations	Create/list/read/update/delete and download PNG/SVG/PDF.
/qr/scans	Decode images, analyze content, list/delete history.
/qr/access	Read protected-link policy and unlock.
/analytics	Filtered analytics overview.
/bulk	Create/list/read/delete jobs and download ZIP output.
/shares	Owner CRUD/history plus public policy, grant, and download.
/dashboard	Cross-feature personal summary.
/notifications	Inbox, unread state, deletion, and preferences.
/admin	Overview, paginated users, access changes, audit data.
/health, /meta	Liveness/readiness and runtime capabilities/limits.

Dynamic redirects are exposed separately at `/r/{slug}`. Complete request/response schemas are always available from `/openapi.json`, `/docs`, and `/redoc`.

11. Testing, build, and troubleshooting

Validation commands

```
# Backend
cd C:\codes\react\IntelijQR\backend
.\.venv\Scripts\Activate.ps1
python -m pytest

# Frontend
cd C:\codes\react\IntelijQR\frontend
npm test
npm run typecheck
npm run build

# Preview the production frontend build
npm run preview
```

Verified repository status on 10 August 2026

- **55 backend tests passed** with one third-party Starlette/httpx deprecation warning.
- **24 frontend tests passed** across 11 Vitest files.
- **TypeScript compilation and Vite production build passed** (1,848 modules transformed).

Common problems

Symptom	Resolution
Backend settings validation fails	Create backend/.env from the example and provide secrets of at least 32 characters.
Readiness returns 503 / operations fail	Start MongoDB and verify URI/database settings. Enable required-on-startup when failure should stop the API.
Browser cannot call API	Confirm both servers are running, VITE_API_BASE_URL is correct, and the frontend origin is listed in backend CORS settings.
Phone-scanned QR cannot redirect	Set redirect/frontend base URLs to the computer's LAN IP; 127.0.0.1 on a phone refers to the phone itself.
Protected data cannot decrypt	Restore the exact original vault encryption key. Do not casually rotate this key.
PowerShell blocks activation	Use .\.venv\Scripts\python.exe directly, or change execution policy only according to local security rules.
Email is not delivered	Run a local SMTP capture service on port 1025, enable user preferences, and keep the host as localhost.

Summary: IntelliQR combines a typed React client with a layered FastAPI/MongoDB backend. Its strongest architectural themes are stable dynamic QR links, explicit feature boundaries, local-first processing, owner isolation, encrypted access workflows, and automated contract validation.